

一步步自己写出来教程

结论

这份代码不要靠死背。真正掌握的方法是按模块重写，每写完一个模块就验证一次。

第 1 步：先写公共定义

先写 robot_def.h:

- 定义整车模式 ROBOT_STOP / ROBOT_NORMAL
- 定义底盘模式 CHASSIS_INDEPENDENT / CHASSIS_SPIN / CHASSIS_FOLLOW_GIMBAL
- 定义电机编号
- 写 Robot_LimitFloat()
- 写 Robot_WrapAngleDeg()

你要能解释：为什么角度要 wrap 到 -180° 到 180° 。

第 2 步：写 PID

先只写 pid.c，不要碰底盘和云台。

测试方法：

```
PID_Init(&pid, 1, 0, 0, -1000, 1000, -100, 100);  
output = PID_Calc(&pid, 100, 80, 0.005f);
```

如果 $k_p=1$ ，误差是 20，输出应接近 20。

第 3 步：写按键输入

写 remote.c:

- 消抖
- 短按
- 长按

- 长按连发

先用调试变量观察，不要接电机。

第 4 步：写底盘运动学

先只写目标轮速，不写电机输出。

输入：

`vx, vy, wz`

输出：

`wheel_target[4]`

观察变量是否符合预期。

第 5 步：接底盘 PID

底盘闭环链路：

目标轮速 - 编码器反馈速度 -> PID -> 电机输出

没有编码器时，只能说“接口完成”，不能说“闭环实测完成”。

第 6 步：写云台目标角

先不接电机，只让 key2/key3 改变目标角。

重点：

- yaw 目标角要 wrap
- pitch 目标角要限位

第 7 步：接 IMU 反馈

云台闭环链路：

目标角 - IMU 反馈角 -> PID -> 云台输出

注意：不能用电机编码器角度冒充 IMU 姿态角。

第 8 步：写安全模块

把急停、遥控丢失、限幅状态放进 `safety.c`。

你要能说清楚：

- 为什么默认 STOP
- 为什么急停要复位 PID
- 为什么 Pitch 限位要限制目标角，而不是只限制输出

第 9 步：写 BSP 硬件接口

最后才改 `bsp_port.c`：

- 按键 GPIO
- LED
- PWM 或 CAN 电机输出
- 编码器速度
- IMU 角度

第 10 步：上板测试顺序

1. 只测 LED。
2. 只测按键。
3. 只看目标速度和目标角。
4. 电机悬空，低输出测试方向。
5. 接编码器，看速度反馈方向。
6. 接 PID，小参数慢慢调。
7. 接 IMU，测试云台角度。
8. 最后落地跑车。

Source: C:/Temp/codex_rm_assignment/rm_control_assignment_extracted.md:90